



ESPE

UNIVERSIDAD DE LAS FUERZAS ARMADAS

INNOVACIÓN PARA LA EXCELENCIA

PERIODO : 2026 Abril– Agosto 2026

ASIGNATURA : Aplicaciones Distribuidas

TEMA : Sistema distribuido de
gestión bomberil

ESTUDIANTE : Joseph Franco

NIVEL-PARALELO : 7mo

DOCENTE : Ing. Geovanny Brito

FECHA DE ENTREGA: 29/07/2026

SANTO DOMINGO - ECUADOR

ÍNDICE

Resumen.....	6
Introducción	6
Descripción del problema	6
Justificación	7
Objetivos	7
Objetivo general.....	7
Objetivos específicos	7
Arquitectura final del sistema	8
Tecnologías utilizadas.....	9
Marco teórico	9
Orquestación y réplicas con Docker Swarm	9
Comunicación síncrona, tiempo real y asincronía	10
Autenticación multifactor	10
Almacenamiento híbrido e integridad.....	10
Inteligencia artificial local	10
Desarrollo e implementación del sistema	11
Clúster y ejecución distribuida.....	11
Interfaz y experiencia de usuario	13

Flujo operativo y consistencia	17
Módulo Premium y PayPal Sandbox	18
Correo OTP y TOTP	20
Almacenamiento privado con PocketBase.....	21
Integración IoT ESP32 post-incendio	22
Integración de inteligencia artificial local.....	25
Integración con contenidos anteriores	26
Pruebas realizadas	27
Resultados obtenidos	28
Problemas encontrados y soluciones aplicadas.....	28
Matriz de cumplimiento del enunciado.....	29
Conclusiones finales	29
Recomendaciones	30
Referencias bibliográficas.....	30
Anexos	31
Anexo A. Repositorio y archivos de ejecución.....	31
Anexo B. Comandos de verificación	31
Anexo C. Correspondencia de pines del prototipo IoT.....	32

ÍNDICE DE FIGURAS

Figura 1. <i>Arquitectura distribuida final de la plataforma.</i>	8
Figura 2. <i>Estado verificado del stack Docker Swarm.</i>	11
Figura 3. <i>Contenedores y tareas reales administrados por Docker Swarm.</i>	12
Figura 4. <i>Inicio de sesión unificado de la plataforma.</i>	13
Figura 5. <i>Formulario ciudadano de reporte georreferenciado.</i>	14
Figura 6. <i>Unidad operativa en espera de despacho.</i>	15
Figura 7. <i>Atención de emergencia, mapa e indicadores IoT.</i>	16
Figura 8. <i>Flujo completo desde el reporte hasta el cierre.</i>	17
Figura 9. <i>Captura de datos para la instalación preventiva.</i>	18
Figura 10. <i>Flujo de creación y captura del pago Sandbox.</i>	19
Figura 11. <i>Almacenamiento híbrido de evidencia multimedia.</i>	21
Figura 12. <i>Registro real de evidencia en PocketBase.</i>	21
Figura 13. <i>Prototipo virtual del nodo táctico en Wokwi.</i>	23
Figura 14. <i>Implementación física del nodo IoT post-incendio.</i>	24
Figura 15. <i>Monitoreo de telemetría IoT en la interfaz del operador.</i>	25

Resumen

El proyecto consolida una plataforma distribuida para gestión de emergencias bomberiles. El ciudadano registra incidentes con geolocalización y evidencia; el administrador recibe la alerta en tiempo real y despacha una unidad operada por personal autenticado; el operador navega al sitio, gestiona el estado y documenta el cierre. El sistema integra Angular 21.2, Spring Boot 4.0.6, PostgreSQL, STOMP sobre WebSocket, Docker Swarm, PocketBase, PayPal Sandbox, autenticación reforzada, Ollama con llama3.2 y un nodo IoT ESP32 para evaluación post-incendio.

La solución se despliega con dos réplicas del frontend y dos del backend, mientras la base de datos permanece en una sola instancia para centralizar la consistencia. Las pruebas ejecutadas registraron 12 casos exitosos en el backend y 7 en el frontend. El almacenamiento multimedia se separó de la base relacional y se añadió integridad SHA-256. Como limitación técnica, la asincronía actual emplea un ThreadPoolTaskExecutor con cola en memoria; no existe todavía un broker persistente externo.

Palabras clave: aplicaciones distribuidas, Docker Swarm, WebSocket, ESP32, PocketBase, PayPal Sandbox, TOTP, Ollama, PostgreSQL.

Introducción

La gestión de una emergencia requiere que información, recursos y decisiones se coordinen con la menor latencia posible. Este proyecto integra en una sola solución el reporte ciudadano, el despacho central, la operación de unidades, el cierre documental, el monitoreo post-incendio y servicios complementarios de prevención. La arquitectura distribuye responsabilidades sin fragmentar la experiencia del usuario.

La arquitectura responde a un caso realista. Los reportes ciudadanos son variables e impredecibles; las unidades deben asignarse una sola vez; los estados tienen que propagarse sin recargar la página; la evidencia debe conservar su integridad; y la lectura de aire post-incendio se asocia con el informe operativo. La distribución se utiliza donde aporta disponibilidad y se conserva una única fuente de verdad transaccional.

Descripción del problema

Los procedimientos tradicionales de atención dependen de llamadas, radio y actualización manual. Esto dificulta recibir evidencia visual y ubicación precisa, identificar qué unidades tienen realmente un operador disponible, evitar asignaciones simultáneas y conservar una bitácora verificable. Después de un incendio también existe incertidumbre sobre la calidad del aire, cuya observación suele quedar separada del reporte.

Desde el punto de vista informático, una aplicación centralizada en un solo proceso se convierte en un punto único de fallo. Al mismo tiempo, replicar servicios sin coordinar estado, WebSocket, archivos y transacciones puede generar inconsistencias. El problema consiste en diseñar una plataforma que distribuya la carga y se recupere de la caída de tareas, pero mantenga una sola verdad operacional.

Justificación

La plataforma aporta valor social porque reduce pasos entre el ciudadano y la central, mejora la visibilidad del administrador y guía al operador. Académicamente, permite aplicar REST, WebSocket, concurrencia, JWT, OAuth 2.0, transacciones, réplicas, almacenamiento externo, pagos Sandbox, autenticación multifactor, IA local e IoT dentro de un caso cohesionado de Aplicaciones Distribuidas.

Objetivos

Objetivo general

Ampliar y validar la aplicación distribuida de gestión bomberil mediante clúster, comunicación síncrona y en tiempo real, seguridad multifactor, almacenamiento privado, pagos de prueba, inteligencia artificial local y telemetría IoT, conservando la consistencia operativa del sistema.

Objetivos específicos

Desplegar frontend y backend como servicios replicados de Docker Swarm y verificar su estado deseado.

Integrar PayPal Sandbox para el plan Premium de prevención, validando monto, moneda, orden y captura.

Implementar OTP por correo y TOTP compatible con aplicaciones autenticadoras.

Guardar evidencias en PocketBase con metadatos relacionales e integridad SHA-256.

Vincular la telemetría del ESP32 al reporte del operador y a la bitácora final.

Incorporar un copiloto local Ollama limitado a información operativa autorizada.

Ejecutar pruebas automatizadas y documentar de forma transparente las brechas pendientes.

Arquitectura final del sistema

La Figura 1 presenta la topología consolidada. El routing mesh de Swarm expone los servicios replicados, Spring Boot concentra las reglas de negocio y PostgreSQL actúa como fuente transaccional. PocketBase, Ollama, PayPal, correo y ESP32 se integran mediante interfaces controladas.

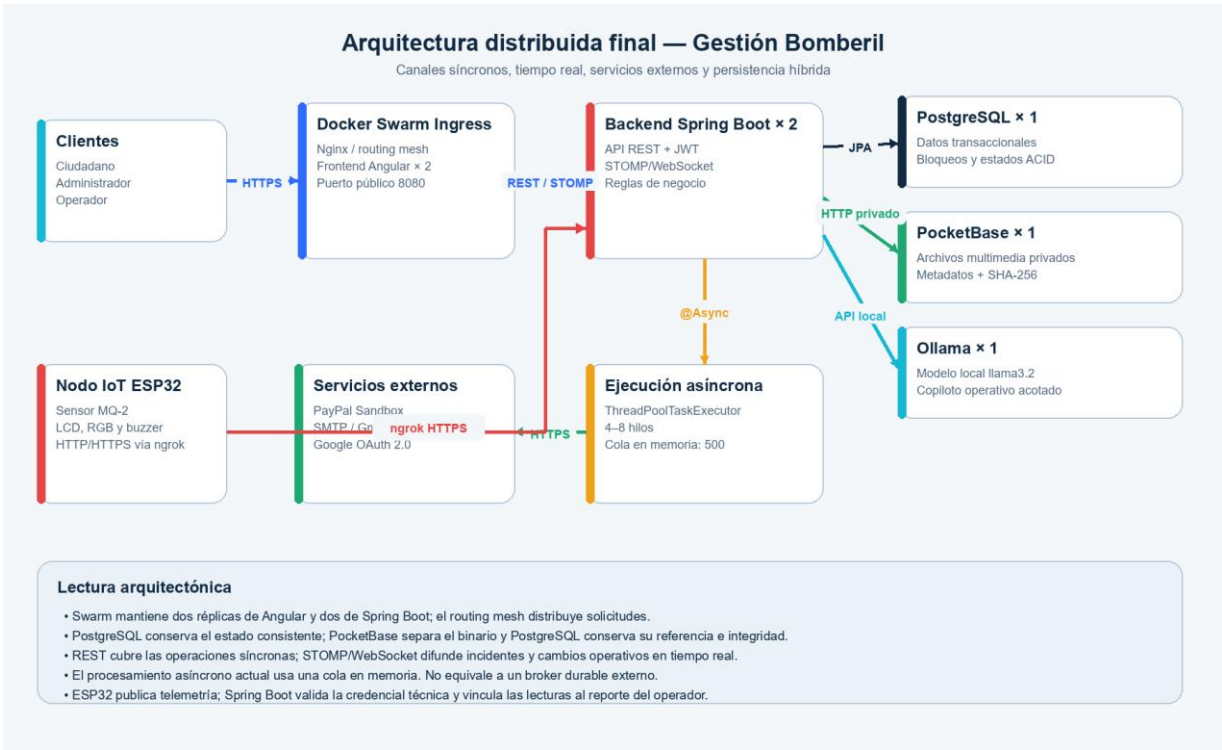


Figura 1. Arquitectura distribuida final de la plataforma.

La decisión de mantener una instancia de PostgreSQL responde al enunciado y evita divergencias entre réplicas. El backend no conserva sesión HTTP local: autentica mediante JWT y persiste el estado compartido. No obstante, el broker STOMP configurado es simple y vive dentro de cada réplica. Para compensar esta limitación en el prototipo, el cliente reconcilia periódicamente el estado por REST; en producción se recomienda un relay STOMP externo.

Capa	Componente	Instancias	Responsabilidad
Presentación	Angular 21.2 + Nginx	2	Interfaces de ciudadano, administrador y operador.
Negocio	Spring Boot 4.0.6	2	REST, seguridad, transacciones, STOMP e integraciones.
Datos	PostgreSQL 17	1	Estado transaccional y trazabilidad.

Capa	Componente	Instancias	Responsabilidad
Archivos	PocketBase	1	Binarios multimedia privados.
IA	Ollama / llama3.2	1	Copiloto operativo local.
IoT	ESP32 + MQ-2	1 nodo	Telemetría de humo/gas post-incendio.
Exposición	ngrok	1	Túnel HTTPS para el dispositivo IoT.

Tecnologías utilizadas

Tecnología	Versión / modalidad	Uso en el proyecto
Angular	21.2	SPA, componentes, guards, formularios y cliente STOMP.
Spring Boot	4.0.6 / Java 25	API REST, JPA, seguridad, correo, pagos e IoT.
PostgreSQL	17 Alpine	Persistencia relacional ACID.
Docker Swarm	Modo Swarm	Orquestación, réplicas, red overlay e ingress.
PocketBase	Contenedor local	Archivos y registros de evidencia.
PayPal	Orders API v2 Sandbox	Creación, aprobación y captura de pago.
Ollama	llama3.2 local	Resumen y consultas operativas acotadas.
ESP32	DOIT DevKit V1	Adquisición analógica y envío HTTPS.
STOMP/WebSocket	Broker simple + SockJS	Difusión en vivo de incidentes y estados.

Marco teórico

Orquestación y réplicas con Docker Swarm

Swarm administra servicios mediante un estado deseado. Cada réplica corresponde a una tarea independiente; si una tarea termina, el orquestador crea otra para recuperar el número configurado. El routing mesh acepta solicitudes en el puerto publicado y las conduce hacia una tarea activa. En este proyecto la disponibilidad frente a la caída de un contenedor queda

demostrada, pero el clúster posee un solo nodo físico; por tanto, no tolera el apagado total del equipo anfitrión.

Comunicación síncrona, tiempo real y asincronía

REST implementa el patrón solicitud–respuesta para comandos y consultas que requieren confirmación inmediata. STOMP sobre WebSocket mantiene un canal bidireccional y publica eventos a suscriptores. El envío de correo se ejecuta mediante `@Async` y un pool de 4 a 8 hilos con capacidad de 500 tareas. Esta cola es volátil: mejora la latencia del usuario, pero no entrega reintentos durables ni confirmaciones propias de RabbitMQ o Kafka.

Autenticación multifactor

OTP verifica posesión temporal del correo mediante un código de seis dígitos y vigencia limitada. TOTP deriva el valor a partir de un secreto compartido y el tiempo, con pasos de 30 segundos según RFC 6238. El sistema genera el secreto, presenta el QR, confirma la vinculación y solicita el código durante el inicio de sesión.

Almacenamiento híbrido e integridad

Separar el binario de los datos transaccionales reduce el crecimiento de PostgreSQL. PocketBase conserva el archivo, mientras PostgreSQL mantiene el identificador, MIME, tamaño, reporte y SHA-256. El hash no cifra la imagen: permite detectar cambios y demostrar correspondencia entre registro y contenido.

Inteligencia artificial local

Ollama ejecuta llama3.2 dentro del entorno local, evitando enviar el contexto operacional a una API externa. El backend construye un contexto permitido, clasifica la intención, aplica límites y controla timeouts. Las respuestas son asistencia para el administrador y no sustituyen las decisiones de despacho.

Desarrollo e implementación del sistema

Clúster y ejecución distribuida

La Figura 2 resume la comprobación realizada sobre el stack: dos réplicas de frontend, dos de backend y una instancia para cada servicio con estado. También se verificaron los endpoints de salud sin exponer secretos.

Estado verificado del stack Docker Swarm			
Servicio	Réplicas	Función	Puerto local
 frontend	2 / 2	Angular + Nginx	8080
 backend	2 / 2	Spring Boot	8082 → 8081
 postgres	1 / 1	PostgreSQL 17	5433 → 5432
 pocketbase	1 / 1	Almacenamiento	8091 → 8090
 ollama	1 / 1	llama3.2 local	11435 → 11434
 ngrok	1 / 1	Túnel IoT HTTPS	4041 inspector
Verificación funcional frontend HTTP 200 · backend UP · PocketBase 200 · Ollama 200 · ngrok 200 Nota: es un clúster de un solo nodo físico; las réplicas toleran caída de tareas, no la caída del equipo anfitrión.			

Figura 2. Estado verificado del stack Docker Swarm.

La Figura 3 corresponde a la captura proporcionada durante la ejecución. Los indicadores verdes muestran las tareas activas del frontend y backend, mientras las filas detenidas conservan

el historial de tareas reemplazadas. Esta vista permite explicar que Swarm crea una tarea nueva cuando una réplica termina.



ID	Nombre	CPU	Memoria	Replicación
6b0345ffc85b	gestion-bor	0%	0B / 0B	0
7c54d46a7e72	gestion-bor	0%	0B / 0B	0
52fd48f712e4	gestion-bor	0.17%	357.7MB / 7.39GI	54
6ecea9f9df72	gestion-bor	0.11%	339.3MB / 7.39GI	53
c4281f540955	gestion-bor	0%	0B / 0B	0
613698478cf9	gestion-bor	0%	0B / 0B	0
44e0bad30c45	gestion-bor	0%	12.04MB / 7.39GI	13
8291b059fdec	gestion-bor	0%	10.79MB / 7.39GI	13
-	proyecto_partes	0%	0B / 0B	0
-	practica-socket	0%	0B / 0B	0

Figura 3. Contenedores y tareas reales administrados por Docker Swarm.

Servicio	Puerto del host	Puerto interno	Acceso
Frontend	8080	80	http://localhost:8080
Backend	8082	8081	http://localhost:8082/api/health
PostgreSQL	5433	5432	Solo administración local.
PocketBase	8091	8090	http://localhost:8091/_/
Ollama	11435	11434	API local del modelo.
ngrok inspector	4041	4040	http://localhost:4041

El Algoritmo 6 presenta la secuencia operativa utilizada. Swarm no reinicia el mismo contenedor: reemplaza la tarea terminada por otra hasta recuperar el número de réplicas definido.

Algoritmo 6. Despliegue reproducible del stack.

```
# Preparar variables sin publicar secretos
.\scripts\configurar-docker.ps1

# Construir imágenes y desplegar
.\scripts\iniciar-swarm.ps1

# Verificar estado deseado
.\scripts\estado-swarm.ps1
docker service ls

# Prueba de autorrecuperación
docker service ps gestion-bomberil_backend
docker kill <contenedor_de_una_tarea>
docker service ps gestion-bomberil_backend
```

Interfaz y experiencia de usuario

La Figura 4 evidencia la pantalla de acceso. El diseño permite autenticación con Google o credenciales de operador y mantiene una identidad visual común en modo claro y oscuro.

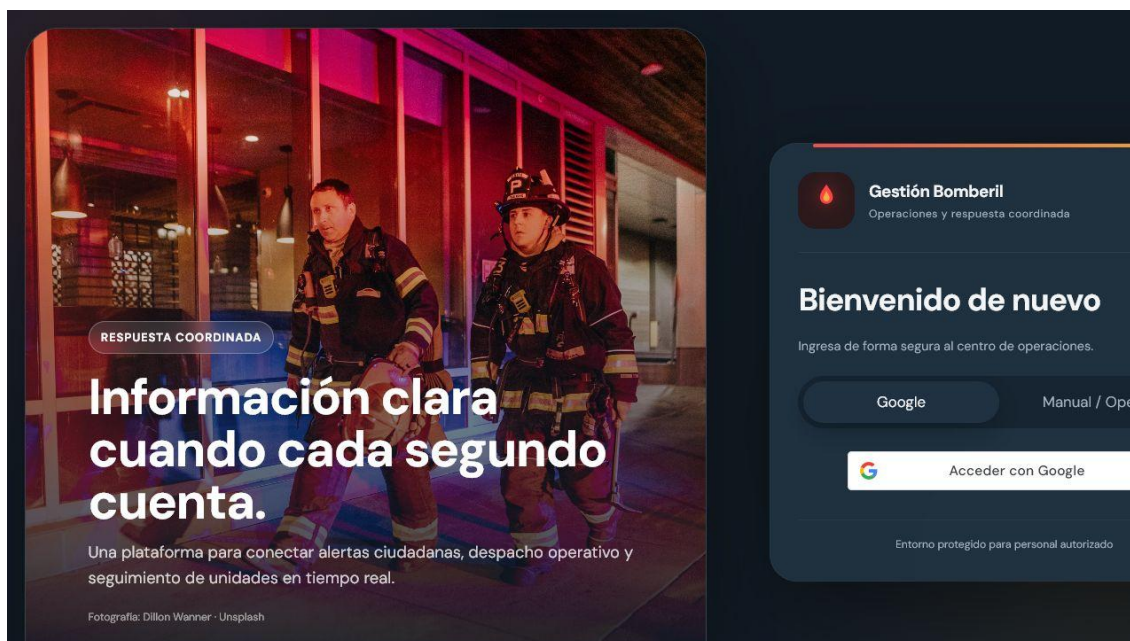


Figura 4. Inicio de sesión unificado de la plataforma.

La Figura 5 muestra el flujo público para seleccionar el tipo de emergencia, describirla y confirmar la ubicación. Las categorías visuales reducen errores de clasificación y la validación impide continuar con datos incompletos.

Ayuda ante emergencias
Canal ciudadano de Gestión Bomberos

Paso 1 de 4 **PLAN PREMIUM**
Preservación de datos para tu establecimiento

¿Qué tipo de emergencia reportas?

Selecciona la categoría que mejor describe la situación

Incendio **Accidente** **Gas / Quím.** **Derrumbe**
Inundación **Otro**

Descripción de la emergencia

Describe brevemente lo que está ocurriendo. Incluye detalles importantes como número de personas, estructuras afectadas, etc...

300 caracteres restantes

Ubicación de la emergencia

Ubicación detectada
-0.253012 - -79.177024
Obteniendo coordenadas exactas del dispositivo...

Ubicación exacta del siniestro (Se obtiene automáticamente por el GPS de tu dispositivo)

Siguiendo: Evidencia

Figura 5. *Formulario ciudadano de reporte georreferenciado.*

La Figura 6 presenta la vista del operador después de seleccionar una unidad. El estado de disponibilidad se mantiene vinculado a una sesión activa y el canal de despacho informa si la comunicación está disponible.

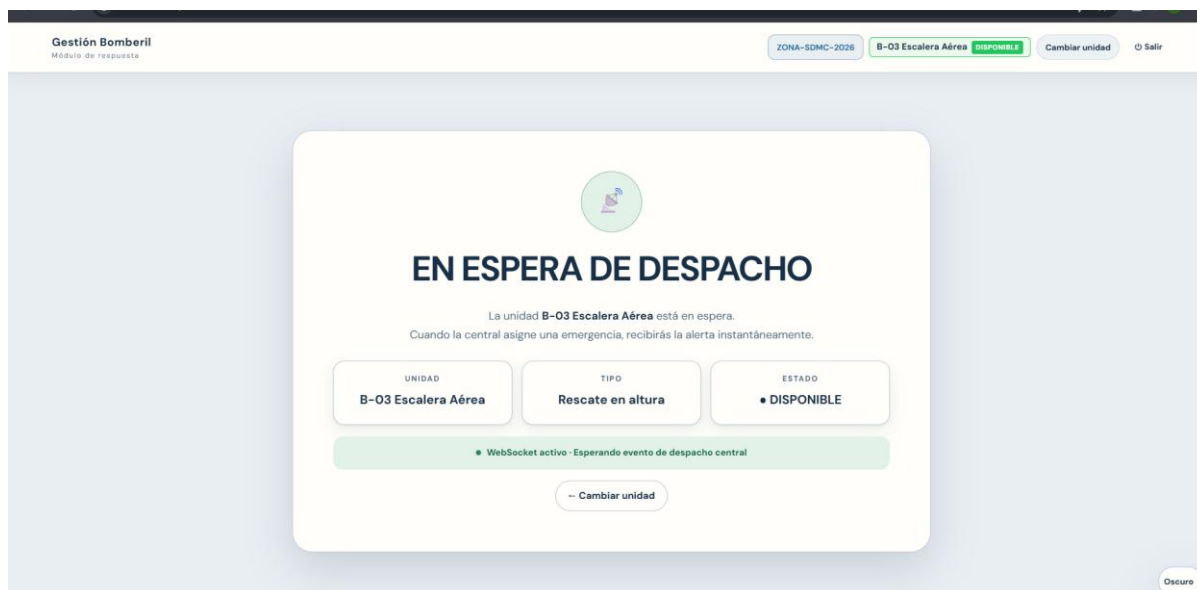


Figura 6. *Unidad operativa en espera de despacho.*

La Figura 7 evidencia la interfaz durante una asignación: mapa, información del incidente, estado operativo y métricas de gas. El operador puede finalizar y liberar la unidad cuando completa la bitácora.

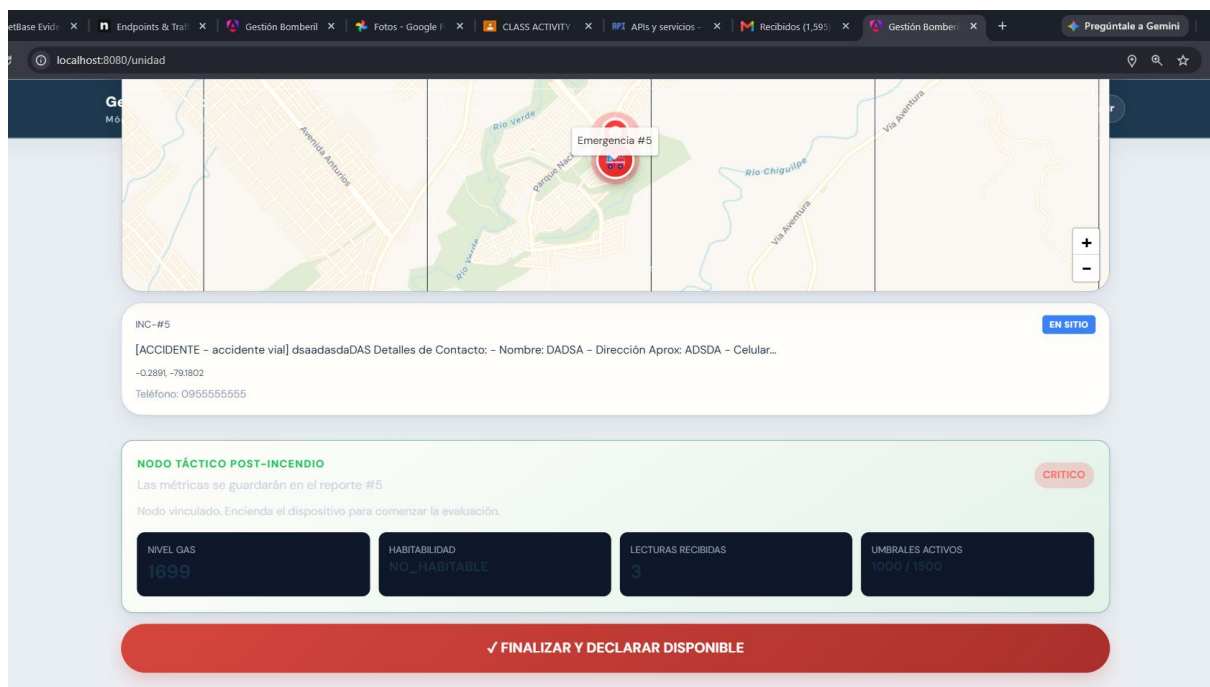


Figura 7. Atención de emergencia, mapa e indicadores IoT.

Flujo operativo y consistencia

La Figura 8 integra los pasos funcionales del sistema. Cada transición persiste primero el estado válido y después publica la notificación; así, una reconexión puede recuperar la verdad desde PostgreSQL.



Figura 8. *Flujo completo desde el reporte hasta el cierre.*

El Algoritmo 7 abstrae el mecanismo de exclusión aplicado. Los bloqueos pesimistas y la transacción evitan que dos administradores asignen simultáneamente la misma unidad o vuelvan a despachar un reporte atendido.

Algoritmo 7. *Despacho transaccional de una unidad.*

```

@Transactional
despachar(reporteId, unidadId):
    reporte = bloquearReporte(reporteId)
    unidad = bloquearUnidad(unidadId)
    validar(reporte.estado == PENDIENTE)
    validar(unidad.estado == DISPONIBLE)
    validar(unidad.operadorActivo)
    reporte.estado = EN_ATENCION
    unidad.estado = EN_RUTA
    guardar(reporte, unidad)
    publicarEventoWebSocket(reporte, unidad)
    return asignacion
  
```

Entidad	Estados principales	Regla crítica
Reporte	PENDIENTE → EN_ATENCION → ATENDIDO	ATENDIDO no puede volver a despacharse.
Unidad	DISPONIBLE → EN_RUTA → EN_SITIO → DISPONIBLE	Solo se ofrece si existe operador activo.
IoT Evaluación	ACTIVA → FINALIZADA	Un nodo no se vincula a dos evaluaciones activas.
Orden Premium	PENDIENTE → APROBADA/PAGADA o FALLIDA	Monto y moneda se validan en servidor.

Módulo Premium y PayPal Sandbox

La Figura 9 muestra el registro del establecimiento previo al pago. Los datos permiten coordinar una visita técnica y no se consideran pagados hasta recibir y validar la captura de PayPal.

1 Datos del lugar — 2 Pago seguro — 3 Confirmación

🏠 ¿Dónde instalaremos el servicio?

Un profesional utilizará estos datos para coordinar la visita.

Nombre completo *

asdas

Correo electrónico *

jafranco5@espe.edu.ec

Teléfono / WhatsApp *

0991234567

Tipo de establecimiento *

Vivienda

Nombre del establecimiento

Opcional

Dirección exacta *

EL ORO 101 Y LA RIA

Ciudad *

Santo Domingo

Provincia *

Santo Domingo de los Tsáchilas

Referencia para llegar

Color del inmueble, sector, punto de referencia...

📍 Ubicación aproximada detectada para facilitar la visita técnica.

Atrás Continuar al pago →

Figura 9. Captura de datos para la instalación preventiva.

La Figura 10 detalla el proceso del plan Premium. La orden interna se vincula con la orden de PayPal, y la confirmación final se almacena solo después de conciliar la respuesta del proveedor.

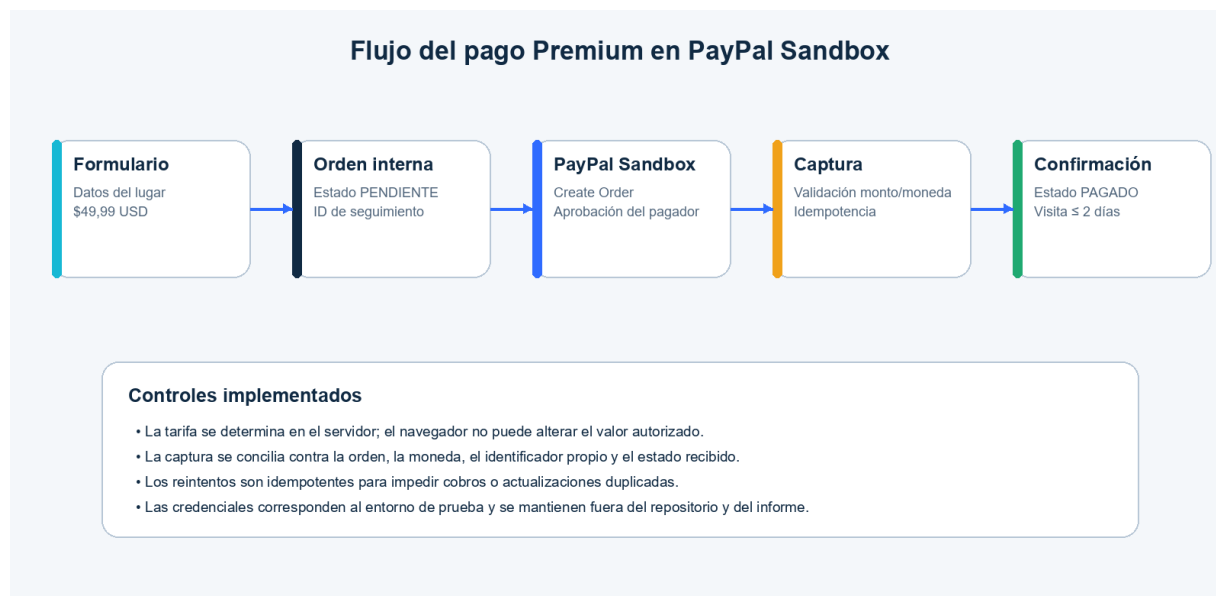


Figura 10. *Flujo de creación y captura del pago Sandbox.*

El Algoritmo 8 resume la defensa contra manipulación de precio y reintentos. La implementación observada registra la orden y el resultado, aunque el proyecto todavía requiere una vista administrativa específica con filtros, historial de cambios y reembolsos.

Algoritmo 8. *Confirmación idempotente de pago.*

```

confirmarPago(ordenInterna, paypalOrderId):
  validar(ordenInterna.paypalOrderId == paypalOrderId)
  if ordenInterna.estado == PAGADO:
    return ordenInterna
  respuesta = capturarEnPayPal(paypalOrderId)
  validar(respuesta.moneda == "USD")
  validar(respuesta.monto == precioServidor)
  validar(respuesta.customId == ordenInterna.codigo)
  if respuesta.estado == "COMPLETED":
    ordenInterna.estado = PAGADO
    ordenInterna.instalacionMaxima = hoy + 2 dias
    guardarAuditoriaOperativa(ordenInterna)
    return ordenInterna
  
```

Correo OTP y TOTP

El correo OTP se compone mediante una plantilla HTML y se envía en segundo plano. El código tiene seis dígitos y expira después de diez minutos. Para TOTP, el backend genera un secreto Base32, construye el QR, confirma la primera clave y valida ventanas de 30 segundos con tolerancia controlada.

El Algoritmo 9 representa la validación del segundo factor. La ventana ± 1 reduce falsos rechazos por una pequeña diferencia de reloj. La desactivación autoservicio de TOTP no fue identificada y queda como mejora.

Algoritmo 9. Validación TOTP durante el inicio de sesión.

```

validarTotp(usuario, codigo):
    validar(usuario.mfaHabilitado)
    tiempoActual = epochSeconds() / 30
    for desplazamiento in [-1, 0, 1]:
        esperado = HOTP(usuario.secreto, tiempoActual + desplazamiento)
        if comparacionConstante(esperado, codigo):
            emitirJWT(usuario)
            return AUTORIZADO
    registrarIntentoFallido(usuario)
    return CODIGO_INVALIDO_O_VENCIDO

```

Control	Estado	Evidencia técnica
OTP por correo	Implementado	Código de 6 dígitos, plantilla HTML, expiración y envío @Async.
Recuperación de contraseña	Pendiente	No se identificó enlace firmado de recuperación.
Adjuntos de correo	Pendiente	No se identificó MimeMessage con archivos adjuntos.
TOTP activar/validar	Implementado	Secreto, QR, confirmación y validación en login.
TOTP desactivar	Pendiente	Debe añadirse endpoint protegido y reautenticación.

Almacenamiento privado con PocketBase

La Figura 11 explica la separación del binario y sus metadatos. El navegador consume el endpoint autorizado de Spring Boot y no utiliza directamente la credencial técnica de PocketBase.



Figura 11. Almacenamiento híbrido de evidencia multimedia.

La Figura 12 evidencia una imagen almacenada con SHA-256, identificador de reporte, MIME y tamaño. El token mostrado durante la configuración nunca se incorpora al documento ni al repositorio.

Id	archivo	T sha256	# reportId	T mimeType	# tamanoBytes	created	updated
evvh307s96f7uud		415add4f45af69a80753d0f4c1a47a31ea5e...	114	image/jpeg	15160	2024-07-28 17:25:51.135Z	2024-07-28 17:25:51.135Z

Figura 12. Registro real de evidencia en PocketBase.

El Algoritmo 10 presenta la escritura y lectura controlada. Las reglas bloqueadas evitan una consulta anónima del registro; como endurecimiento adicional se recomienda marcar el campo como archivo protegido y usar tokens de corta duración o mantener exclusivamente el proxy actual.

Algoritmo 10. *Carga e integridad de la evidencia.*

```

    guardarEvidencia(reporteId, archivo):
        validarMimeYTamano(archivo)
        hash = SHA256(archivo.bytes)
        registroPB = pocketBase.crear(
            archivo, reporteId, hash, mime, tamano)
        evidenciaSQL = nueva Evidencia(
            reporteId, registroPB.id, hash, mime, tamano)
        repositorio.guardar(evidenciaSQL)
        return evidenciaSQL

consultarEvidencia(id, usuario):
    validarRolYAcceso(usuario, id)
    return backend.descargarDesdePocketBase(id)

```

Integración IoT ESP32 post-incendio

El nodo táctico se diseñó para colocarse después de mitigar un incendio estructural. El sensor MQ-2 entrega una lectura analógica; el firmware clasifica RESPIRABLE, PRECAUCIÓN o CRÍTICO, actualiza LCD y LED RGB, y activa el buzzer cuando se supera el umbral peligroso. En paralelo publica telemetría con nodo, valor y evento.

La Figura 13 muestra la simulación del circuito: ESP32, sensor MQ-2, pantalla I2C, LED RGB, pulsador y buzzer. La simulación permitió verificar pines, estados visuales y umbrales antes de energizar el montaje real.

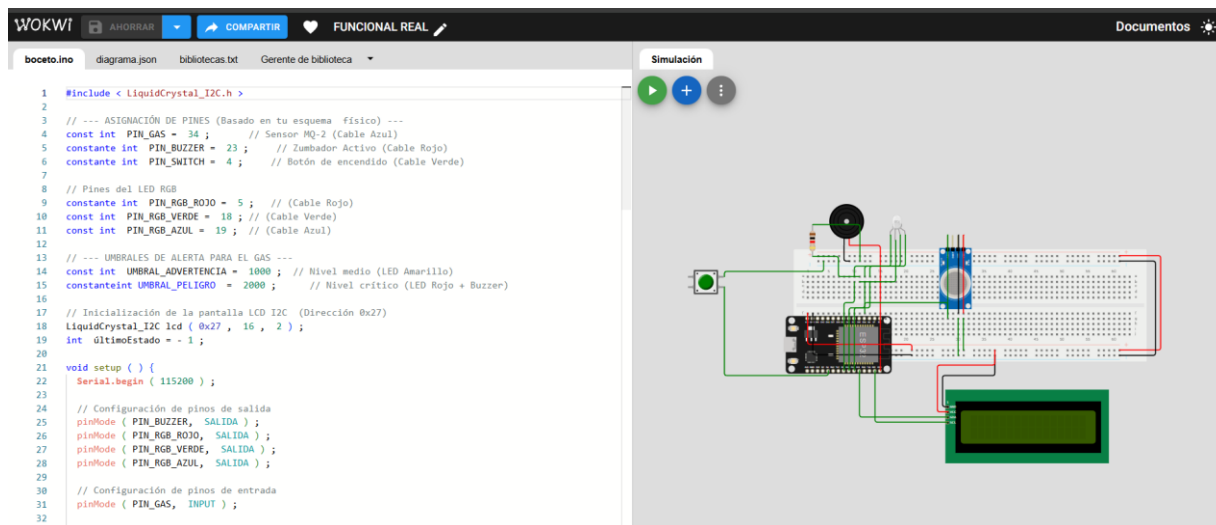


Figura 13. *Prototipo virtual del nodo táctico en Wokwi.*

La Figura 14 presenta el prototipo físico funcional con ESP32, protoboard, sensor, indicadores y LCD integrados en una carcasa de demostración. Esta evidencia confirma el paso de la simulación al dispositivo real.



Figura 14. *Implementación física del nodo IoT post-incendio.*

La Figura 15 muestra la recepción de gas y el estado de transmisión. Para evitar estados falsos, la interfaz considera conectado al nodo únicamente cuando existe telemetría reciente; si expira el intervalo, informa desconexión aunque las lecturas históricas permanezcan visibles.

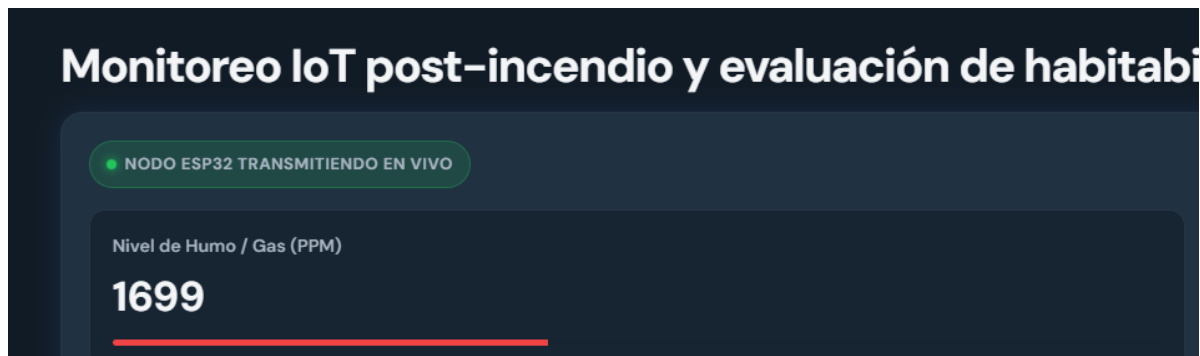


Figura 15. *Monitoreo de telemetría IoT en la interfaz del operador.*

El Algoritmo 11 evita bloquear el ciclo con una alarma basada en `millis()`. Los umbrales residen en el firmware y el mismo estado calculado se envía al backend, lo que impide que dispositivo e interfaz interpreten valores con reglas diferentes.

Algoritmo 11. *Publicación periódica de telemetría desde ESP32.*

```

loop():
  gas = analogRead(PIN_GAS)
  estado = clasificar(gas, UMBRAL_ADVERTENCIA, UMBRAL PELIGRO)
  actualizarLCD_RGB_Buzzer(estado)
  if sistemaEncendido and millis() - ultimoEnvio >= intervalo:
    POST /api/iot/telemetria
    headers: X-IOT-KEY
    body: { nodoId, nivelGas: gas, evento: "TELEMETRIA" }
    registrarCodigoHTTP()
    reconectarWiFiSiCorresponde()

```

Integración de inteligencia artificial local

El Copiloto Operativo consulta únicamente datos del sistema autorizados: emergencias activas, unidades, operadores, despachos, bitácoras y monitoreo IoT. Las preguntas fuera de dominio se rechazan. Para evitar una respuesta puramente determinista, el backend recupera contexto validado y construye un prompt acotado que Ollama transforma en una explicación legible.

Puede responder	No debe responder
Unidades disponibles y su tipo.	Programación general o tareas académicas ajenas.
Resumen del turno y emergencias activas.	Datos personales no necesarios para la operación.
Estado reciente del nodo IoT y habitabilidad.	Instrucciones que alteren seguridad o credenciales.
Estado de despachos y cierres operativos.	Información no contenida en el contexto del sistema.

El Algoritmo 12 reduce alucinaciones al consultar primero el estado real y limitar el modelo. Si Ollama excede el tiempo de espera o devuelve una respuesta inválida, la API entrega un error controlado y no bloquea el dashboard.

Algoritmo 12. *Consulta acotada del Copiloto Operativo.*

```

responder(pregunta, administrador):
    validarJWTyRol(administrador)
    intencion = clasificarIntencion(pregunta)
    if intencion not in DOMINIO_OPERATIVO:
        return respuestaFueraDeAlcance()
    contexto = consultarDatosMinimos(intencion)
    prompt = construirPromptSistema(contexto, pregunta)
    respuesta = ollama.chat("llama3.2", prompt, timeout)
    validarRespuesta(respuesta)
    return sanitizar(respuesta)

```

Integración con contenidos anteriores

Contenido conservado	Aplicación actual
Cliente-servidor	Angular consume una API Spring Boot desacoplada.
WebSocket	STOMP/SockJS publica reportes, despachos, cierres e IoT.
Concurrencia	Pool asíncrono y bloqueos pesimistas en asignación.
Excepciones	Manejador global devuelve errores JSON consistentes.
Logs	Eventos de autenticación, despacho, IoT, correo y pagos.
JWT / OAuth 2.0	Rutas protegidas y autenticación Google.
Roles	Ciudadano público; operador y administrador autenticados.
SOLID	Controladores delgados, servicios cohesionados y repositorios.
Patrones	Repository, Service Layer, DTO, Strategy de almacenamiento y Observer pub/sub.

Contenido conservado	Aplicación actual
Transacciones	Estados operativos, asignación, cierre y pago.

El diseño sigue el principio de responsabilidad única: los controladores traducen HTTP, los servicios aplican reglas y los repositorios encapsulan persistencia. EvidenceStorageService actúa como abstracción de almacenamiento y permite seleccionar PocketBase o almacenamiento local mediante configuración. Las integraciones externas se encapsulan para que sus errores no contaminen directamente la lógica de presentación.

Pruebas realizadas

Nivel	Prueba	Resultado
Backend	Maven test: controladores, contexto, Ollama y servicios de unidades.	12/12 aprobadas.
Frontend	Angular/Karma: componentes y servicios.	7/7 aprobadas.
Clúster	Estado de servicios y réplicas.	2/2 frontend, 2/2 backend; demás 1/1.
Salud	Frontend, backend, PocketBase, Ollama y ngrok.	HTTP 200 / backend UP.
Persistencia	Conteos y estados en PostgreSQL.	Datos consistentes y consultables.
IoT	WiFi, DNS, HTTPS, autenticación técnica y telemetría.	Flujo funcional con clave coincidente.

La base consultada durante la verificación contenía 6 reportes ciudadanos, 6 unidades, 2 evidencias multimedia, 1 orden Premium y 85 lecturas IoT. Cuatro reportes estaban ATENDIDOS y dos PENDIENTES; la orden registrada figuraba PAGADA. Estas cifras son una instantánea de prueba y pueden variar con nuevas ejecuciones.

APROBADO	Backend: 12 pruebas, sin fallos, errores ni omisiones.
-----------------	--

APROBADO	Frontend: 7 pruebas distribuidas en 4 archivos, todas exitosas.
-----------------	---

VERIFICADO	Servicios del stack operativos y endpoints de salud accesibles.
-------------------	---

Resultados obtenidos

El administrador recibe y gestiona reportes; el operador obtiene la asignación sin recargar gracias a eventos y reconciliación.

Solo se ofrecen para despacho unidades con operador activo y estado DISPONIBLE.

El cierre publica el resultado en tiempo real, libera la unidad y bloquea un nuevo despacho del reporte ATENDIDO.

Las evidencias se almacenan fuera de PostgreSQL y conservan hash SHA-256 y referencia relacional.

El pago Sandbox genera una orden Premium y programa la instalación después de la captura válida.

La telemetría real del ESP32 se incorpora al reporte con primera y última toma de aire.

Ollama responde dentro del dominio operacional y utiliza el estado del sistema como contexto.

Problemas encontrados y soluciones aplicadas

Problema	Causa	Solución aplicada / recomendada
Asignación no aparecía hasta recargar.	Eventos entre réplicas con broker simple.	WebSocket más reconciliación REST y eventos de cierre.
IoT mostraba conectado sin transmitir.	Se usaba la existencia de lecturas históricas.	Estado por antigüedad de la última telemetría.
HTTP 401 del ESP32.	Clave técnica distinta entre firmware y stack.	Unificar IOT_NODE_KEY y reiniciar el servicio.
HTTPS 421 / conexión rechazada.	Ruta/túnel/red y SNI.	Diagnóstico DNS/TLS, URL HTTPS y túnel correcto.
OAuth origin_mismatch.	Origen localhost no autorizado.	Registrar el origen exacto en Google Cloud.
Multimedia ausente.	Configuración/token PocketBase y referencias.	Proveedor PocketBase, colección, proxy y SHA-256.
Réplicas 0/2 temporalmente.	Base de datos no saludable o recursos.	Healthchecks, orden de arranque y revisión de tareas.

Matriz de cumplimiento del enunciado

Requisito	Estado	Observación verificable
Docker Swarm y réplicas	Cumple	Frontend 2, backend 2 y PostgreSQL 1.
REST síncrono	Cumple	46 endpoints mapeados aproximadamente.
Cola de mensajes externa	Parcial	Existe cola en memoria; falta RabbitMQ/Kafka durable.
Pagos Sandbox	parcial Cumple	Pago y conciliación listos; falta consola admin completa/auditoría de cambios.
OTP por correo	Cumple	HTML, expiración y ejecución @Async.
Recuperación y adjuntos	Pendiente	No fueron identificados en el código revisado.
TOTP	parcial Cumple	Alta y validación listas; falta desactivación autoservicio.
PocketBase privado	Cumple	Acceso vía backend, reglas bloqueadas, SHA-256.
Inteligencia artificial	Cumple	Ollama llama3.2 con dominio acotado.
Soft delete, paginación y filtros	Parcial	Estados conservan operaciones; faltan patrones uniformes en todos los listados.
README y .env.example	Cumple	Guías Docker y ejemplo sin secretos.

La matriz evita presentar como terminadas funciones que no aparecen en la implementación. Para alcanzar el 100 % del enunciado se debe incorporar un broker durable, recuperación de contraseña, adjuntos de correo, desactivación TOTP, consola administrativa de transacciones con auditoría y una estrategia uniforme de borrado lógico/paginación.

Conclusiones finales

El proyecto integra sus componentes en un ecosistema distribuido funcional. La plataforma coordina actores, datos relacionales, archivos, pagos, IA e IoT sin perder el ciclo operacional del incidente. El uso de réplicas demuestra autorrecuperación frente a fallos de tareas y el empleo de una sola base conserva una fuente de verdad coherente.

La integración ESP32 constituye el aporte más tangible: las proyecciones del informe anterior se materializaron en un dispositivo físico que mide el ambiente post-incendio y entrega sus datos al informe del operador. La comparación Wokwi-prototipo confirma una metodología incremental de simulación, construcción y validación.

La revisión también evidencia que distribuir no significa únicamente añadir contenedores. WebSocket con broker local, una cola volátil y un clúster de un solo nodo tienen límites claros.

Reconocerlos permite proponer una evolución responsable hacia RabbitMQ, relay STOMP, múltiples nodos físicos, observabilidad y respaldos coordinados.

Recomendaciones

Incorporar RabbitMQ como cola durable y como relay STOMP compartido entre réplicas.

Añadir un segundo nodo físico al Swarm para tolerar la pérdida del anfitrión.

Completar recuperación de contraseña, adjuntos HTML y desactivación TOTP protegida.

Crear una consola administrativa de pagos con filtros, auditoría, conciliación y reembolsos.

Uniformar borrado lógico, restauración, paginación y filtros en usuarios, pagos y archivos.

Agregar métricas Prometheus/Grafana, trazas y alertas de salud.

Calibrar el MQ-2 con condiciones controladas; no usar el prototipo como certificación profesional de habitabilidad.

Referencias bibliográficas

Docker, Inc. (2026). How services work. <https://docs.docker.com/engine/swarm/how-swarm-mode-works/services/>

Docker, Inc. (2026). Use Swarm mode routing mesh. <https://docs.docker.com/engine/swarm/ingress/>

Spring. (2026). WebSocket STOMP overview. <https://docs.spring.io/spring-framework/reference/web/websocket/stomp/overview.html>

Spring. (2026). Task execution and scheduling. <https://docs.spring.io/spring-framework/reference/integration/scheduling.html>

PayPal. (2026). Orders API v2. <https://developer.paypal.com/docs/api/orders/v2/>

PocketBase. (2026). Files upload and handling. <https://pocketbase.io/docs/files-handling/>

PocketBase. (2026). API rules and filters. <https://pocketbase.io/docs/api-rules-and-filters/>

M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). TOTP: Time-Based One-Time Password Algorithm (RFC 6238). <https://datatracker.ietf.org/doc/html/rfc6238>

Fette, I., & Melnikov, A. (2011). The WebSocket Protocol (RFC 6455). <https://datatracker.ietf.org/doc/html/rfc6455>

Ollama. (2026). API documentation. <https://docs.ollama.com/api>

Angular. (2026). Angular documentation. <https://angular.dev/>

PostgreSQL Global Development Group. (2026). PostgreSQL documentation. <https://www.postgresql.org/docs/>

Anexos

Anexo A. Repositorio y archivos de ejecución

Repositorio: https://github.com/Joseph-Franco692/Proyecto_Emergencias_jf. Los archivos `docker-stack.yml`, `compose.yaml`, `DOCKER_SWARM.md`, `POCKETBASE_EVIDENCIAS.md`, `.env.docker.example` y los scripts de la carpeta `scripts` documentan el despliegue. El archivo `.env.docker` real se mantiene excluido de Git.

Anexo B. Comandos de verificación

El Algoritmo 13 reúne la evidencia reproducible de clúster, salud y pruebas. Ningún comando imprime variables sensibles ni el contenido del archivo de entorno.

Algoritmo 13. *Verificación integral sin exponer secretos.*

```
# Servicios y tareas
docker service ls
docker stack services gestion-bomberil
docker stack ps gestion-bomberil

# Salud
```

```
Invoke-WebRequest http://localhost:8082/api/health
Invoke-WebRequest http://localhost:8091/api/health
Invoke-WebRequest http://localhost:11435/api/tags
```

```
# Pruebas
cd emergencias
mvn test
cd ../central-bomberos
npm test -- --watch=false
```

Anexo C. Correspondencia de pines del prototipo IoT

Elemento	Pin ESP32	Función
MQ-2 AOUT	GPIO 34	Entrada analógica de humo/gas.
Buzzer	GPIO 23	Alarma acústica.
Pulsador	GPIO 4	Encendido/apagado lógico.
RGB rojo	GPIO 5	Estado crítico.
RGB verde	GPIO 19	Estado respirable.
RGB azul	GPIO 18	Sistema en espera/apagado.
LCD SDA	GPIO 21	Bus I2C de datos.
LCD SCL	GPIO 22	Bus I2C de reloj.

Los valores de umbral son parámetros de calibración del prototipo y deben ajustarse con mediciones de referencia. El dispositivo es una demostración académica; no reemplaza instrumentos certificados ni protocolos profesionales para declarar habitable una estructura.